

# Programmatic Interaction with SOLVE-IT

# Possible Use Case Ideas

1. Enable [Your Tool] to use the knowledge base for [Your Use Case].
2. Automate weakness checks for your procedures, guidelines, standards, exam protocols, etc.
3. Automate checks for missed mitigations in your notes and reports.

**We're interested in your input!**

# OBLIGATORY DISCLAIMER

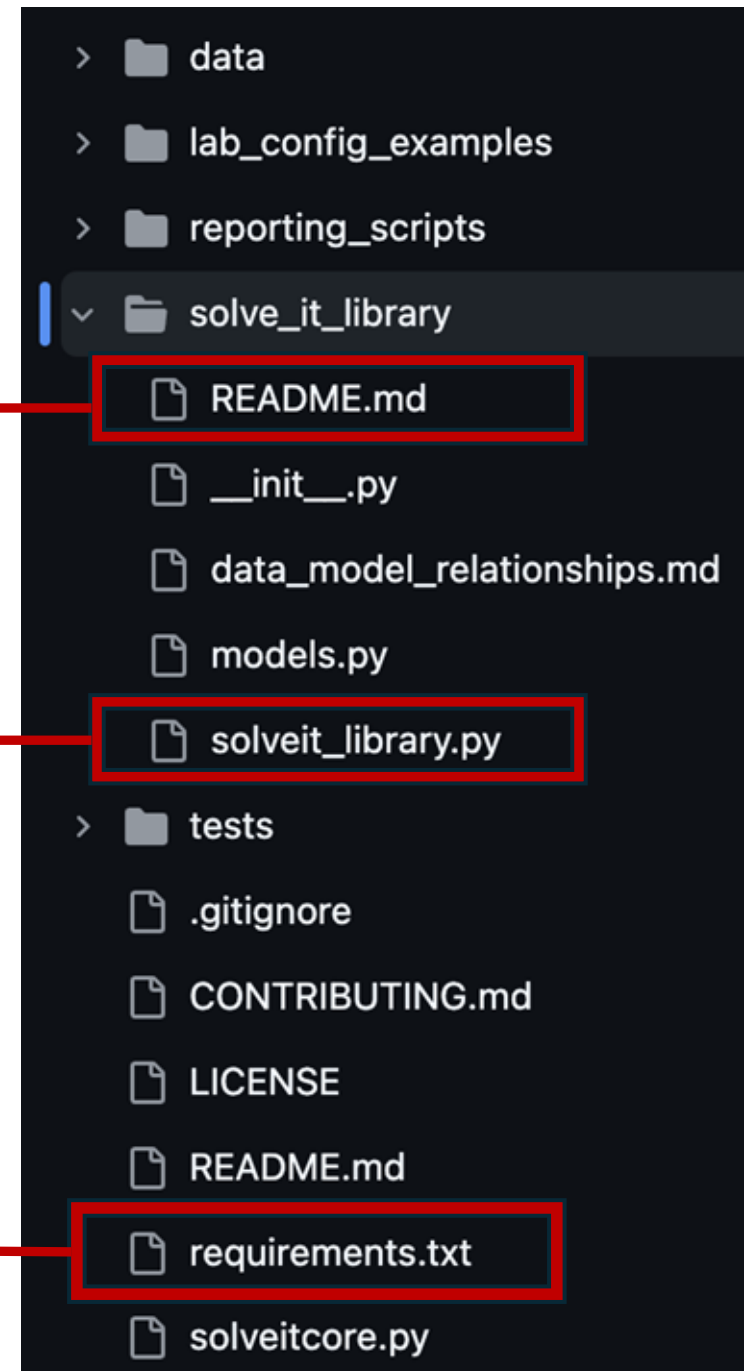
- This is a beta, open-source project. It needs work (Contributions welcome).
- Use with caution in production applications.

# The SOLVE-IT Python Library

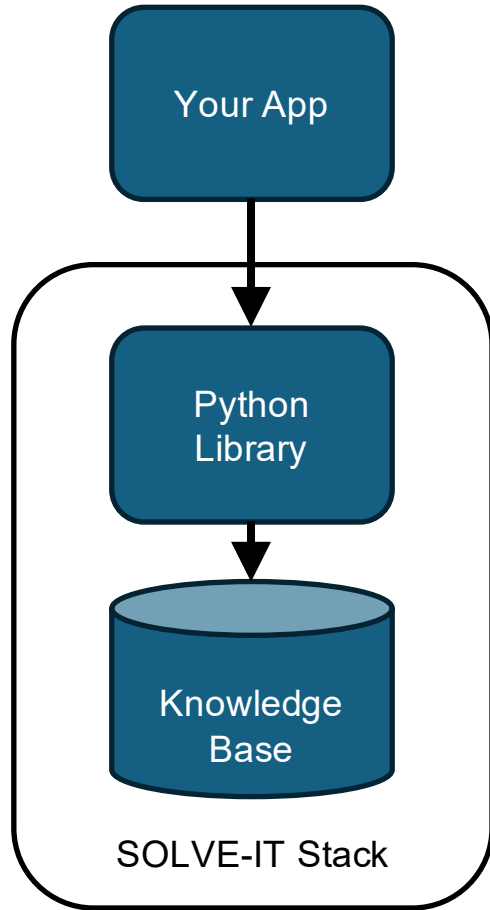
Basic instructions  
for  
library usage.

Main for new  
library.

Requirements up in the  
root of the project.



# Importing and Initializing the Library



1. Your app starts.
2. Your app imports and initializes the library.
3. The library loads the knowledge base.

# Importing and Initializing the Library

```
# Import the KnowledgeBase class
from solve_it_library import KnowledgeBase

# Initialize with path to solve-it repository root (containing 'data' folder)
kb = KnowledgeBase('path-to_solve-it-repo', 'solve-it.json')
```

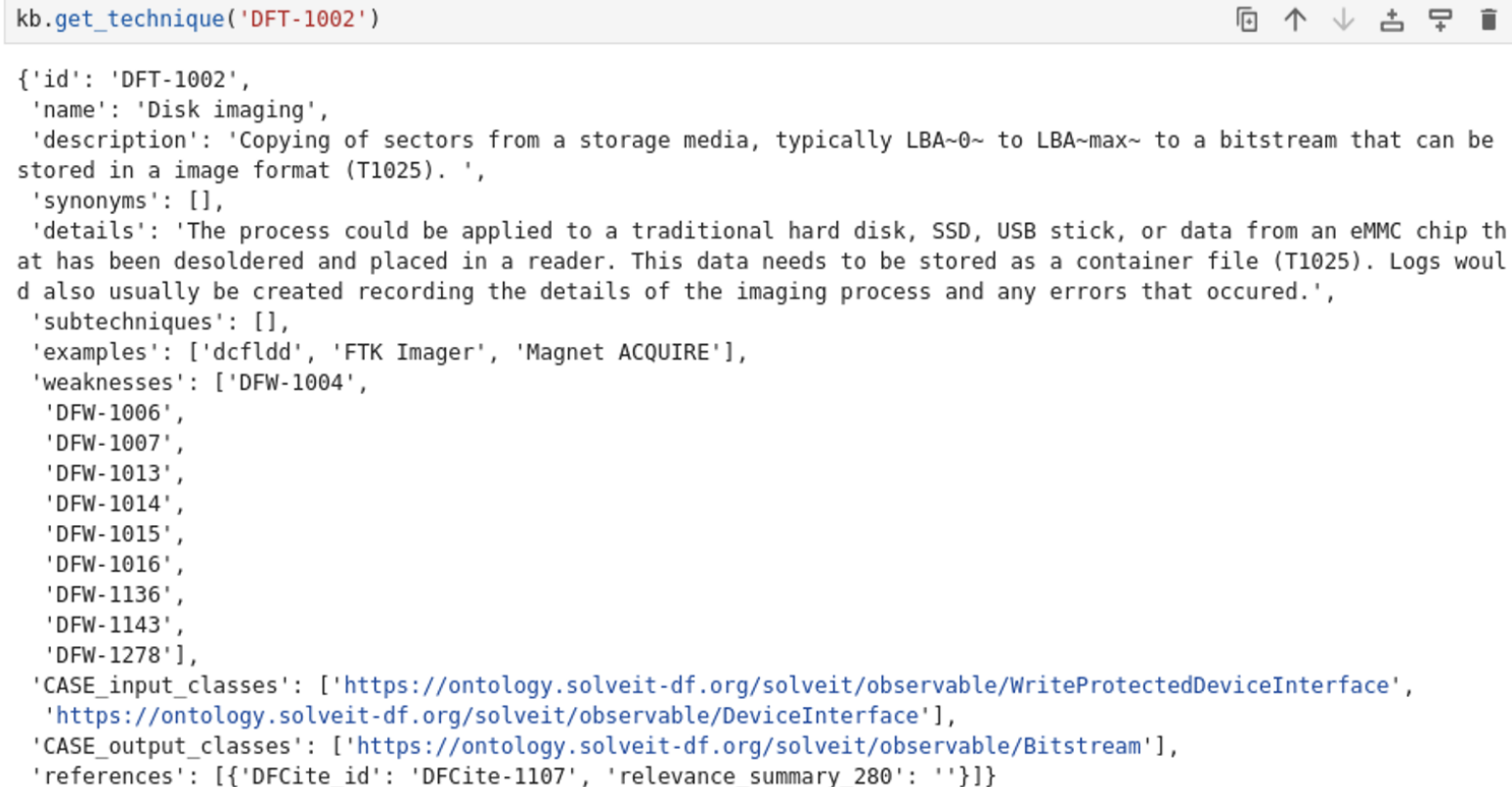
When you initialize the Knowledgebase:

- The library checks for required SOLVE-IT data.
- Data structures are prepared, and techniques, weaknesses, mitigations and objectives are loaded into those structures.
- Reverse lookup indices are built from that data.
- The chosen objective mapping is loaded.
- The library is then ready to serve SOLVE-IT data.

# Calling SOLVE-IT Python Library Functions: Part 1

Use: `kb.function_name(parameters)`

Example: `kb.get_technique("T1002")`



```
kb.get_technique('DFT-1002')

{'id': 'DFT-1002',
 'name': 'Disk imaging',
 'description': 'Copying of sectors from a storage media, typically LBA~0~ to LBA~max~ to a bitstream that can be stored in a image format (T1025). ',
 'synonyms': [],
 'details': 'The process could be applied to a traditional hard disk, SSD, USB stick, or data from an eMMC chip that has been desoldered and placed in a reader. This data needs to be stored as a container file (T1025). Logs would also usually be created recording the details of the imaging process and any errors that occurred.',
 'subtechniques': [],
 'examples': ['dcfldd', 'FTK Imager', 'Magnet ACQUIRE'],
 'weaknesses': ['DFW-1004',
 'DFW-1006',
 'DFW-1007',
 'DFW-1013',
 'DFW-1014',
 'DFW-1015',
 'DFW-1016',
 'DFW-1136',
 'DFW-1143',
 'DFW-1278'],
 'CASE_input_classes': ['https://ontology.solveit-df.org/solveit/observable/WriteProtectedDeviceInterface',
 'https://ontology.solveit-df.org/solveit/observable/DeviceInterface'],
 'CASE_output_classes': ['https://ontology.solveit-df.org/solveit/observable/Bitstream'],
 'references': [{'DFCite_id': 'DFCite-1107', 'relevance_summary_280': ''}]}
```

# Calling SOLVE-IT Python Library Functions: Part 2

```
a_technique = kb.get_technique('DFT-1002')
```

```
print(type(a_technique))
```

```
<class 'dict'>
```

```
print(a_technique)
```

```
{'id': 'DFT-1002', 'name': 'Disk imaging', 'description': 'Copying of sectors from a storage media, typically LBA~0~ to LBA~max~ to a bitstream that can be stored in a image format (T1025). ', 'synonyms': [], 'details': 'The process could be applied to a traditional hard disk, SSD, USB stick, or data from an eMMC chip that has been desoldered and placed in a reader. This data needs to be stored as a container file (T1025). Logs would also usually be created recording the details of the imaging process and any errors that occurred.', 'subtechniques': [], 'examples': ['dcflddd', 'FTK Imager', 'Magnet ACQUIRE'], 'weaknesses': ['DFW-1004', 'DFW-1006', 'DFW-1007', 'DFW-1013', 'DFW-1014', 'DFW-1015', 'DFW-1016', 'DFW-1136', 'DFW-1143', 'DFW-1278'], 'CASE_input_classes': ['https://ontology.solveit-df.org/solveit/observable/WriteProtectedDeviceInterface', 'https://ontology.solveit-df.org/solveit/observable/DeviceInterface'], 'CASE_output_classes': ['https://ontology.solveit-df.org/solveit/observable/DeviceInterface'], 'CASE_relevance_classes': ['https://ontology.solveit-df.org/solveit/observable/DeviceInterface'], 'CASE_relevance_summaries': [{'DFCite_id': 'DFCite-1107', 'relevance_summary': 'This technique is used to create a bitstream of a storage media, typically LBA~0~ to LBA~max~ to a bitstream that can be stored in a image format (T1025). Logs would also usually be created recording the details of the imaging process and any errors that occurred.'}]}
```

Expected output format  
and data described in code  
comments for function.

```
def get_technique(self, technique_id: str) -> Optional[Dict[str, Any]]:
```

Retrieves a specific technique by its ID.

**Args:**

**technique\_id (str):** The ID of the technique (e.g., "DFT-1002").

**Returns:**

Optional[Dict[str, Any]]: The technique data dictionary if found, otherwise None.

000 000 000

```
return self.techniques.get(technique id)
```

# Consuming the Library's Logs

The library does not print logs on its own. It does produce, and you will need to choose how to handle produced records.

1

Your code imports the Python logging module and configures a logging handler.

2

When `logger.<level>` is called by the library, a log record is produced. If you set up the handler, the log can be routed to your preferred destination.

```
self.techniques = self._load_json_files(self.techniques_path, Technique)
logger.info("Loaded %d techniques.", len(self.techniques))
```

```
import logging

logging.basicConfig(level=logging.INFO)

from solve_it_library import KnowledgeBase

kb = KnowledgeBase('.', 'solve-it.json')
```

```
INFO:solve_it_library.solveit_library:Loaded 172 techniques.
INFO:solve_it_library.solveit_library:Loaded 271 weaknesses.
INFO:solve_it_library.solveit_library:Loaded 235 mitigations.
INFO:solve_it_library.solveit_library:Loaded 144 citations.
INFO:solve_it_library.solveit_library:Building reverse indices for performance optimization...
INFO:solve_it_library.solveit_library:Reverse indices built: 271 weakness->technique, 235 mitigation->weakness, 23
5 mitigation->technique
INFO:solve_it_library.solveit_library:Loaded objective mapping 'solve-it.json' with 22 objectives.
INFO:solve_it_library.solveit_library:Loaded 0 extension(s).
INFO:solve_it_library.solveit_library:Loaded global configuration from ./extension_data/global_solveit_config.py
```

# Integrating SOLVE-IT with LLM-enabled Applications

# Use Case Ideas

1. **A tool** for learning, using & developing for SOLVE-IT.
2. **Chatting** with an assistant that communicates about techniques *through the lens of SOLVE-IT*.
3. **Grounding** your LLM-enabled app using SOLVE-IT.
4. **Combination** of the above used to evaluate your procedures, guidelines, standards, exam protocols, notes, and more!

\*Counterpoint: SOLVE-IT has a Python library. If you're trying to use SOLVE-IT in your applications, just use the library. MCP is only for integration with LLMs.

# LLM Grounding

**[G]rounding is the ability to connect model output to verifiable sources of information.** If you provide models with access to specific data sources, then grounding tethers their output to these data and reduces the chances of inventing content. This is particularly important in situations where accuracy and reliability are significant.

[...]

- **Reduces model hallucination [...]**
- **Anchors model responses to your data sources.**
- **Provides auditability [...]**

# Model Context Protocol (MCP)

The Model Context Protocol is an open standard that enables developers to build secure, two-way connections between their data sources and AI-powered tools. The architecture is straightforward: developers can either expose their data through MCP servers or build AI applications (MCP clients) that connect to these servers.

**Ref:** <https://www.anthropic.com/news/model-context-protocol>

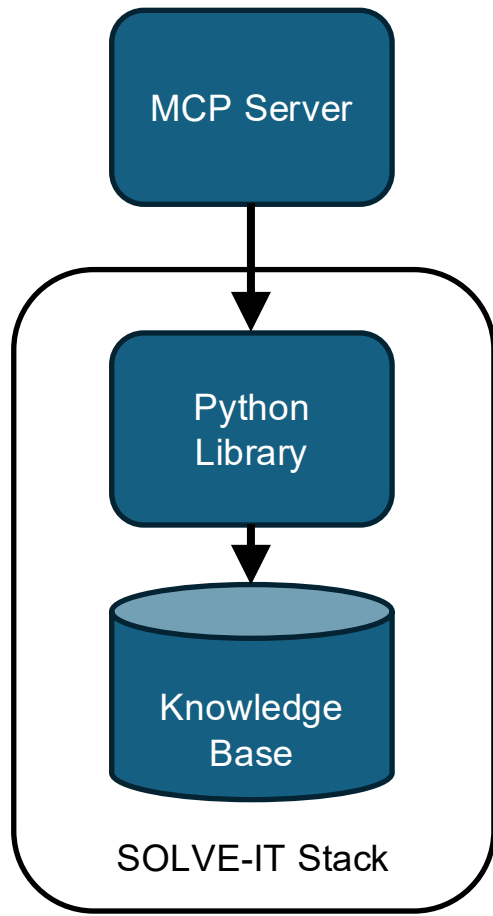
**GitHub:** <https://github.com/modelcontextprotocol>

**Docs:** <https://modelcontextprotocol.io/introduction>

# SOLVE-IT MCP

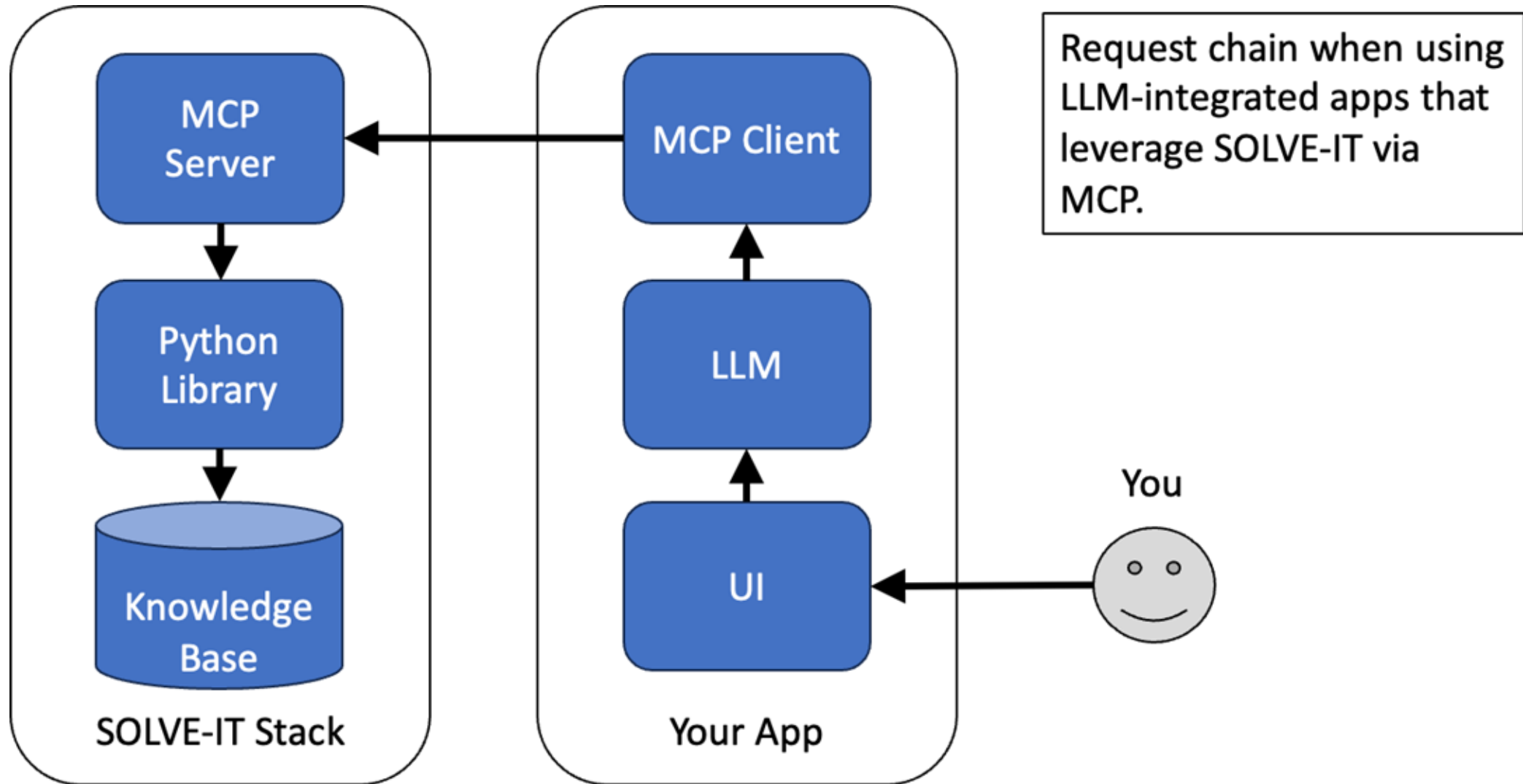
- Server that provides MCP access to the SOLVE-IT library.
- [https://github.com/CKE-Proto/mcp\\_solve-it](https://github.com/CKE-Proto/mcp_solve-it)
- It's not a separate tool. It needs access to the SOLVE-IT Knowledge Base.

# The SOLVE-IT Stack with MCP



1. MCP server starts.
2. MCP server imports and initializes the library.
3. The library loads the knowledge base.

# The SOLVE-IT Stack and Your App



# DISCLAIMER

- **This is a new project. Use at your own risk.**
- Bulk retrieval from SOLVE-IT via MCP and/or multiple, chained requests may result in significant charges to your LLM API accounts. The MCP server has bulk retrieval tools *disabled by default*, but they can be enabled in the config/default.toml file.
- We recommend implementing appropriate API usage metering and rate limits within any applications that leverage this project.
- Privacy! Be mindful of what data you supply to LLMs.

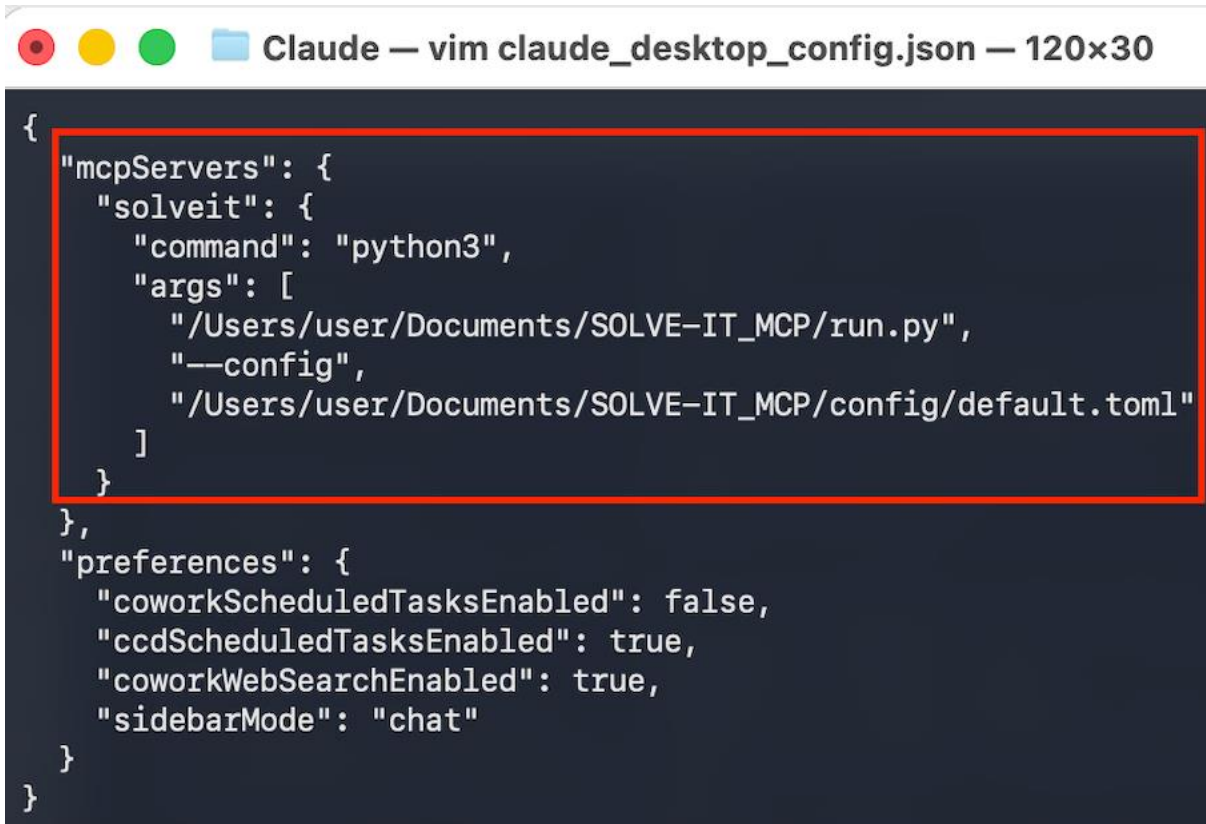
# Setup Process

1. Install dependencies (MCP SDK, pydantic, pybtex)
2. Download MCP Server: [https://github.com/CKE-Proto/solve\\_it\\_mcp](https://github.com/CKE-Proto/solve_it_mcp)
3. Download SOLVE-IT Knowledge Base: <https://github.com/SOLVE-IT-DF/solve-it/>
4. Edit the config/default.toml file to point to the SOLVE-IT repository that you downloaded in Step 2.
5. Configure your LLM client to start and connect to the MCP server.\*

\* If your MCP client does not automatically start the server at connect time (as Claude does), you will also need to manually run the server.

\*\*Review the MCP project README.md for more details.

# Client Config (Claude) - JSON



```
{
  "mcpServers": {
    "solveit": {
      "command": "python3",
      "args": [
        "/Users/user/Documents/SOLVE-IT_MCP/run.py",
        "--config",
        "/Users/user/Documents/SOLVE-IT_MCP/config/default.toml"
      ]
    }
  },
  "preferences": {
    "coworkScheduledTasksEnabled": false,
    "ccdScheduledTasksEnabled": true,
    "coworkWebSearchEnabled": true,
    "sidebarMode": "chat"
  }
}
```

**File:** claude\_desktop\_config.json

**Paths**

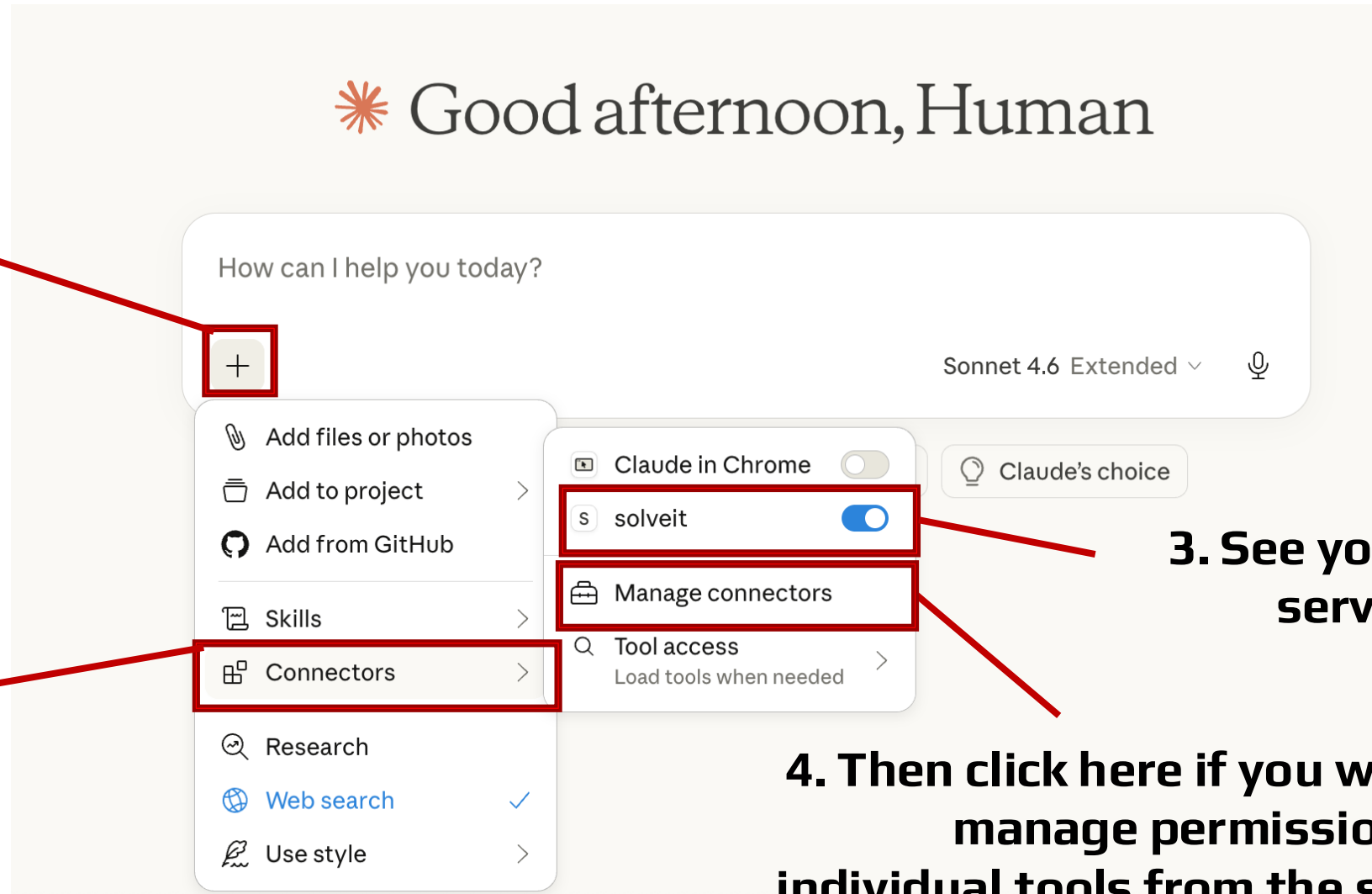
**MacOS:**

/Users/user/Library/Application Support/Claude

# Using the MCP Server w/ Claude Desktop: Part 1 – Observe Server Availability

**1. Click here**

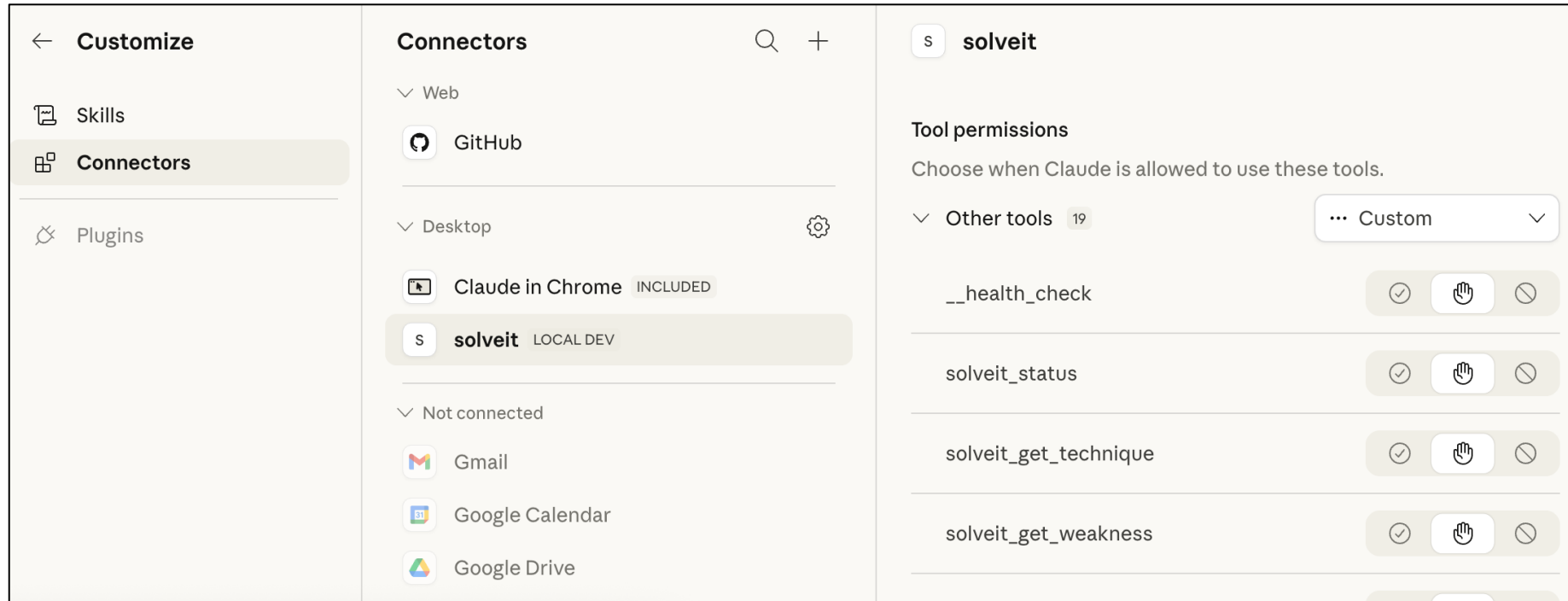
**2. Then click here**



**3. See your MCP server here**

**4. Then click here if you want to manage permissions for individual tools from the server.**

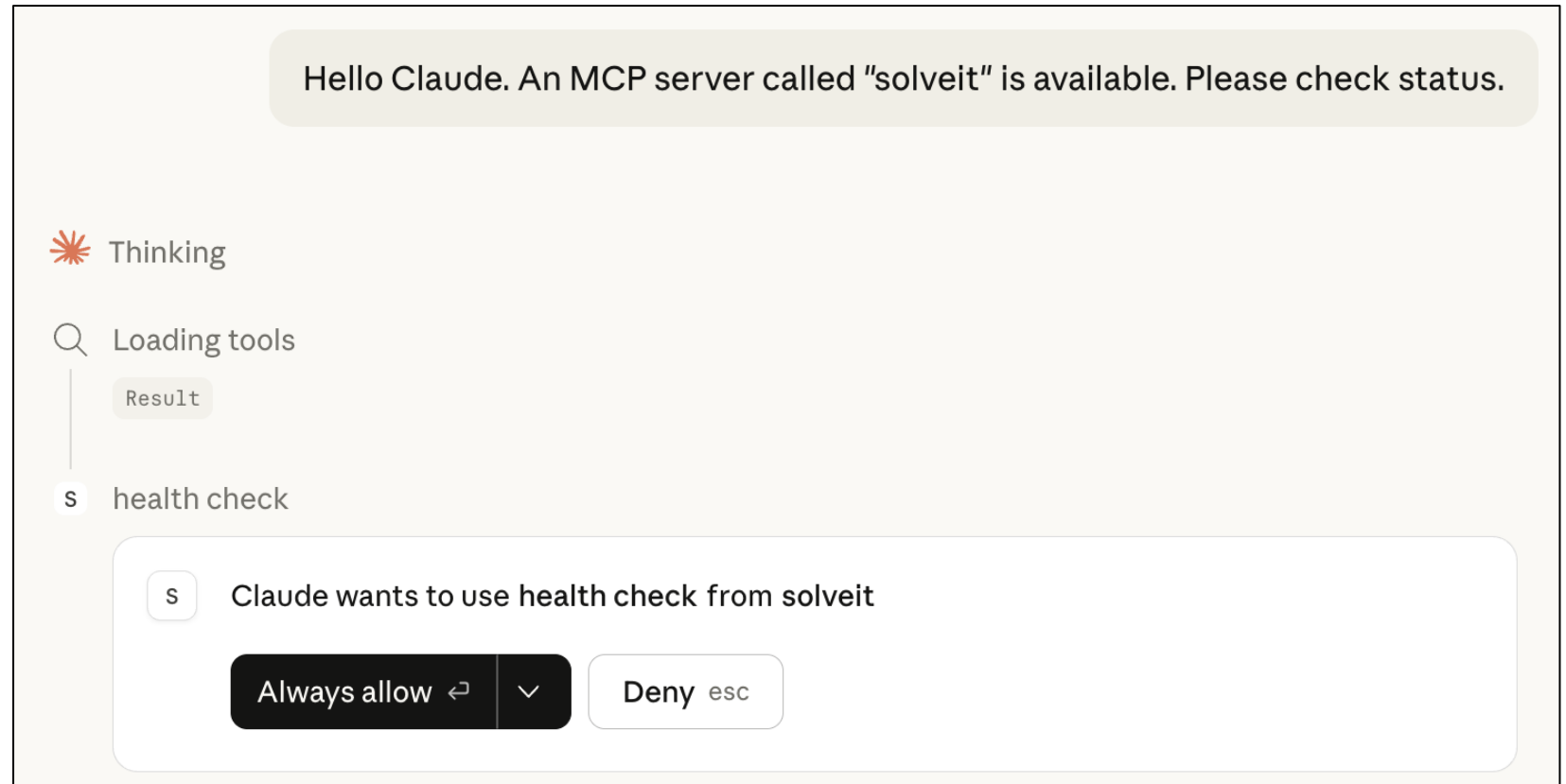
# Using the MCP Server w/ Claude: Part 2 – Observe Tools Under Customize/Connectors



If you were to click that **Manage connectors**, you would see the list of individual tools that it exposes, each of which can be turned on or off, and the permissions for each.

# Using the MCP Server w/ Claude: Part 3 – Approving Tool Usage

**By default, Claude will ask you for permission to use the tool.**



# Using the MCP Server w/ Claude: Part 4 -- Request and Response Visibility

Many LLM clients will show the original request to and response from the MCP server, providing some visibility into the process.

However, you can achieve this with logging from the MCP server itself as well.



The screenshot displays an MCP client interface with a light beige background. At the top, a header reads "Assembled requisite data for technique retrieval" followed by a downward arrow. Below this, a clock icon is next to the text "Let me retrieve the technique and its associated weaknesses." Further down, a small circle with the letter 'S' is next to the text "Solveit get technique". A red arrow points from the text "Many LLM clients will show the original request to and response from the MCP server, providing some visibility into the process." to the "Request" section. Another red arrow points from the text "However, you can achieve this with logging from the MCP server itself as well." to the "Response" section. The "Request" section is a light gray box containing a JSON object: 

```
{  "technique_id": "DFT-1002"}
```

. The "Response" section is a light gray box containing a JSON object: 

```
{"id": "DFT-1002", "name": "Disk imaging", "description": "Copying of sectors from a storage media, typically LBA~0~ to LBA~max~ to a bitstream that can be stored in a"}
```

# Don't expect the LLM to deliver exact, word-for-word content from descriptive fields

Solveit get technique

```
{  
  "id": "DFT-1002",  
  "name": "Disk imaging",  
  "description": "Copying of sectors from a storage media, typically LBA~0~ to LBA~max~ to a bitstream that can be stored in a image format (T1025).",  
  "synonyms": [],  
  "details": "The process could be applied to a traditional hard disk, SSD, USB stick, or data from an eMMC chip that has been desoldered and placed in a reader. This data needs to be stored as a container file (T1025). Logs would also usually be created recording the details of the imaging process and any errors that occurred.",  
  "subtechniques": [],  
  "examples": ["dcfldd", "FTK Imager", "Magnet ACQUIRE"],  
  "weaknesses": ["DFW-1004", "DFW-1006", "DFW-1007", "DFW-1013", "DFW-1014", "DFW-1015", "DFW-1016", "DFW-1136", "DFW-1143", "DFW-1278"]  
}
```

## DFT-1002 — Disk Imaging

**Description:** The copying of sectors from a storage medium, typically from LBA 0 to LBA max, into a bitstream stored in an image format.

**Details:** The technique can be applied to traditional hard disks, SSDs, USB sticks, or data from eMMC chips that have been desoldered and placed in a reader. The resulting data is stored as a container file, and logs are typically created recording the details of the imaging process and any errors encountered.

**Example Tools:** dcfldd, FTK Imager, Magnet ACQUIRE

**CASE Ontology:**

- **Input:** WriteProtectedDeviceInterface, DeviceInterface

**Actual "description" field from T1002**

**Description in response from LLM**

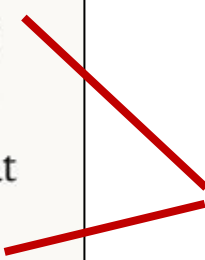
# Do expect the LLM to editorialize

## Technical Details:

- **CASE Output Classes:** observable:Image (follows the Cyber-investigation Analysis Standard Expression format)
- **Reference:** Nikkel, B., 2016. "Practical forensic imaging: securing digital evidence with Linux tools." No Starch Press, Chapter 6, "Forensic Image Acquisition"

This is a fundamental technique in digital forensics, representing the critical first step of creating a forensically sound copy of storage media before analysis begins. The multiple associated weaknesses suggest there are various technical and procedural challenges that investigators need to consider and mitigate when performing disk imaging operations.

Would you like me to explore any of the associated weaknesses to understand what specific challenges or limitations might arise with disk imaging?



**This is all “bonus” content that was unsolicited but supplied by the LLM nonetheless.**

**Remember - by default it wants to be helpful.**

# Related LLM Usage Tips

- Prime the LLM with an understanding of SOLVE-IT by including a description in your initial prompt.
- Leave Bulk Retrieval tools turned off unless you're sure they won't break the bank or trigger your usage limits.
- If you don't want the LLM to editorialize, explicitly tell it not to.